

```

TYPED LIST ARRAY
typedef struct linkedList{
    char data[5];
    int next;
}SIMPLE_LIST;
//This list has at most 100 elements
SIMPLE_LIST L[100];
int first = -1; //Index of the first element
int last = -1; //Index of the last element

POINTER
typedef struct linkedList{
    int info;
    char message[100];
    struct linkedList *next;
}SIMPLE_LIST;
// Initially, the list is empty
SIMPLE_LIST *first = NULL;
SIMPLE_LIST *last = NULL;
int insert(SIMPLE_LIST *p){
    if (first != NULL){
        last->next = p;
        p->next = NULL;
        last = p;
    }
    else {
        first = p;
        last = p;
        p->next = NULL;
    }
    return 0;
}

int displayList(){
    SIMPLE_LIST *p
    p = first;
    if (p == NULL){
        printf("List is empty\n");
        return -1;
    }
    while (p != NULL){
        printf("%d %s\n",
            p->info, p->message);
        p = p->next;
    }
    return 0;
}

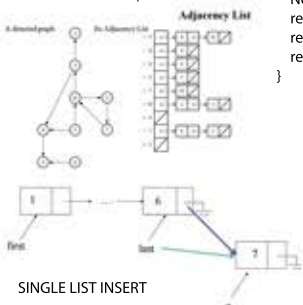
SIMPLE_LIST *search(int key){
    SIMPLE_LIST *p;
    p = first;
    while (p){
        if (p->info == key)
            return p;
        p = p->next;
    }
    return NULL;
}

SIMPLE_LIST *delete(int key){
    SIMPLE_LIST *p, *previous;
    p = first;
    previous = NULL;
    // search for the element to be deleted

    while (p){
        if (key == p->info)
            break;
        previous = p;
        p = p->next;
    }
    if (p != NULL){ //if found
        if (previous == NULL){
            // if first element will be deleted
            if (first == last){
                // if list has one element
                first = NULL;
                last = NULL;
            }
            else {
                first = first->next;
            }
        }
        else {
            //delete from middle or last
            previous->next = p->next;
            if (previous->next == NULL){
                //last element is deleted
                last = previous;
            }
        }
        free(p);
        return(p);
    }
    else //not found
        return NULL;
}

Doubly Linked Lists
typedef struct doubly_list{
    int info;
    char message[100];
    struct doubly_list *previous;
    struct doubly_list *next;
}DLIST;
DLIST *first = NULL;
DLIST *last = NULL;

```



```

DOUBLE LIST
int display(){
    DLIST *p;
    p = first;
    if (p == NULL){
        printf("List is empty\n");
        return -1;
    }
    while (p){
        printf("%d %s\n", p->info, p->message);
        p = p->next;
    }
    return 0;
}

DLIST *search(int key){
    DLIST *p;
    p = first;
    while (p){
        if (key == p->info)
            return p;
        p = p->next;
    }
    return NULL;
}

DLIST *delete(int key){
    DLIST *p;
    p = search(key);
    if (p == NULL){
        printf("The element to be deleted is not in the list\n");
        return NULL;
    }
    if (p == first){ //Delete the first element
        if (first == last){ // list has 1 element
            first = NULL;
            last = NULL;
        }
        else { // list has more than one element
            first = p->next;
            first->previous = NULL;
        }
    }
    else {
        if (p == last){ //Delete from last
            last = p->previous;
            last->next = NULL;
        }
        else { //Delete from middle
            p->previous->next = p->next;
            p->next->previous = p->previous;
        }
        free(p);
        return p;
    }
}

int store(){
    FILE *fp;
    DLIST *p;
    // open the file
    if ((fp=fopen("list.txt","w"))==NULL){
        printf("File cannot be opened, disk is full\n");
        return -1;
    }
    p = first;
    while (p){
        fwrite(p, sizeof(DLIST)-2*sizeof(p), 1, fp);
        p = p->next;
    }
    printf("List was stored\n");
    fclose(fp);
    return 0;
}

1 Counting number of Nodes in Linked List :
int count()
{
    struct node *temp = start;
    int length = 0;
    while(temp!=NULL)
    {
        length++;
        temp=temp->next;
    }
    return(length);
}

RECURSIVE
int count(node *temp)
{
    if(temp == NULL)
        return(0);
    return(1 + count(temp->next));
}

2 copy a linked list, recursively.
struct Node {
    int value;
    Node* next;
};

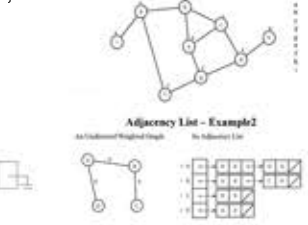
Node* Clone(Node* list) {
    if (list == NULL) return NULL;

    Node* result = new Node;
    result->value = list->value;
    result->next = Clone(list->next);
    return result;
}

Node* Clone(Node* list) {
    if (list == NULL) return NULL;

    Node* result = new Node;
    result->value = list->value;
    result->next = Clone(list->next);
    return result;
}

```



```

COPY
typedef struct slist *LInt;
typedef struct slist{
    int value;
    LInt prox;
}Node;
LInt clone2(LInt l){
    LInt nova=NULL,aux2=NULL;
    while(l){
        aux2=nova;
        nova=(LInt)malloc(sizeof(Node));
        nova->value=l->value;
        nova->prox=aux2;
        l=l->prox;
    }
    return nova;
}

3 Function to reverse the linked list
static void reverse(struct node** head_ref)
{
    struct node* prev = NULL;
    struct node* current = *head_ref;
    struct node* next;
    while (current != NULL)
    {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    *head_ref = prev;
}

RECURSIVE
void recursiveReverse(struct node** head_ref)
{
    struct node* first;
    struct node* rest;
    if (*head_ref == NULL)
        return;
    first = *head_ref;
    rest = first->next;
    if (rest == NULL)
        return 0;
    recursiveReverse(&rest);
    first->next->next = first;
    first->next = NULL;
    *head_ref = rest;
}

time 0(n)
4 remove unsorted linked list
void removeDuplicates(struct node *start)
{
    struct node *ptr1, *ptr2, *dup;
    ptr1 = start;

    /* Pick elements one by one */
    while(ptr1 != NULL && ptr1->next != NULL)
    {
        ptr2 = ptr1;

        while(ptr2->next != NULL)
        {
            if(ptr1->data == ptr2->next->data)
            {
                dup = ptr2->next;
                ptr2->next = ptr2->next->next;
                free(dup);
            }
            else /* This is tricky */
            {
                ptr2 = ptr2->next;
            }
        }
        ptr1 = ptr1->next;
    }
}

Select a Random Node Singly Linked List
void printRandom(struct node *head)
{
    if (head == NULL)
        return;
    srand(time(NULL));

    int result = head->key;

    struct node *current = head;
    int n;
    for (n=2; current!=NULL; n++)
    {
        if (rand() % n == 0)
            result = current->key;
        current = current->next;
    }

    printf("Randomly selected key is %d\n", result);
}

rotate
void rotate(struct node **head_ref, int k)
{
    if (k == 0)
        return;
    struct node* current = *head_ref;
    int count = 1;
    while (count < k && current != NULL)
    {
        current = current->next;
        count++;
    }
    if (current == NULL)
        return;
    struct node *kthNode = current;
    while (current->next != NULL)
        current = current->next;
    current->next = *head_ref;
    *head_ref = kthNode->next;
    kthNode->next = NULL;
}

```

```

bool compareLists(struct node *head1, struct node *head2)
{
    struct node* temp1 = head1;
    struct node* temp2 = head2;
    while (temp1 && temp2)
    {
        if (temp1->data == temp2->data)
        {
            temp1 = temp1->next;
            temp2 = temp2->next;
        }
        else
            return 0;
    }
    if (temp1 == NULL && temp2 == NULL)
        return 1;
    return 0;
}

insert first node
void insert_at_beg()
{
    struct node *new_node,*current;
    new_node=(struct node *)malloc(sizeof(struct node));
    if(new_node == NULL)
        printf("nFailed to Allocate Memory");
    printf("nEnter the data : ");
    scanf("%d",&new_node->data);
    new_node->next=NULL;
    if(start==NULL)
    {
        start=new_node;
        current=new_node;
    }
    else
    {
        new_node->next=start;
        start=new_node;
    }
}

void delete_begin()
{
    temp = start;
    start = start->next;

    free(temp);
    printf("nThe Element deleted Successfully ");
}

STACK-----
ARRAYS
#define N 500
int S[N];
int sp=0;

int push(int item){
    if (sp >= N){
        printf("Stack is full, no insertion is possible\n");
        return -1;
    }
    else {
        S[sp] = item;
        sp++;
        return 0;
    }
}

int pop(){ // remove an element from the stack
    if (sp <= 0){
        printf("Stack is empty\n");
        //assume that stack has positive integers
        return -1;
    }
    else {
        return S[sp];
    }
}

void reset(){ // empty the stack
    sp = 0;
}

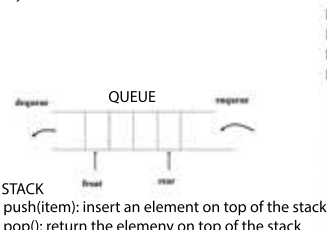
int top(){ // returns the reference to the last
    // inserted element without removing it
    return (sp - 1);
}

int size(){ // returns the # of elements stored
    return sp;
}

int isEmpty(){ // returns 0 if no elements are stored }
    // returns 1 otherwise
    if (sp == 0)
        return 0;
    else
        return 1;
}

EXAMPLE
int main()
{
    char c;
    while ((c=getchar()) !='\n')
        push(c);
    while (sp > 0)
        printf("%c", pop());
    printf("\n");
    return 0;
}

```



```

WITH LINKED LIST
typedef struct s{
    int item;
    struct s *next;
}STACK;
STACK *sp = NULL;
int push(int item){
    STACK *p;
    p = (STACK *)malloc(sizeof(STACK));
    if (p == NULL){
        printf("Error, not enough memory\n");
        return -1;
    }
    else{
        p->item = item;
        p->next = sp;
        sp = p;
        return 0;
    }
}

int pop(){
    STACK *p;
    if (sp == NULL){
        printf("Stack is empty\n");
        return -1;
    }
    else {
        p = sp;
        sp = sp->next;
        free(p);
        return 0;
    }
}

void reset(){
    STACK *p;
    while (sp != NULL){
        p = sp;
        sp = sp->next;
        free(p);
    }
}

STACK *top(){
    return sp;
}
// returns the # of elements in the stack
int i = 0;
p = sp;
while (p){
    i++;
    p = p->next;
}
return i;

STACK *isEmpty(){ // indicates whether no elements
    // are stored or not.
    return sp;
}

palindrome
void push(char);
char pop();
char stack[100];
int top = -1;
void main()
{
    char str[100];
    int i, count = 0, len;
    printf("Enter string to check it is palindrome or not : ");
    scanf("%s", str);
    len = strlen(str);
    for (i = 0; i < len; i++)
        push(str[i]);

    for (i = 0; i < len; i++)
    {
        if (str[i] == pop())
            count++;
    }
    if (count == len)
        printf("%s is a Palindrome string\n", str);
    else
        printf("%s is not a palindrome string\n", str);
}

void push(char c)
{
    stack[++top] = c;
}

char pop()
{
    return(stack[top--]);
}

```

